28. Algorithmic Problem Set (Mixed Drills)

28. Algorithmic Problem Set (Mixed Drills)

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2–3 days
Slug: `28-algorithm-drills`

Learning objectives

- Solve mixed problems under constraints
- State complexity before coding
- Write tests first for edge cases
- Refactor brute force to efficient solutions

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Drill protocol

For each problem: 1. Restate inputs/outputs and constraints 2. Work an example by hand 3. Propose brute force + complexity 4. Improve if needed 5. Implement in Go with table tests 6. Benchmark if n is large

Problem themes in the exercises

- Two-sum variants with maps
- Sliding window
- Interval merge
- Top-K with heaps
- Graph connectivity

Treat this lesson as a gym: correctness under pressure, then clarity.

Common mistakes (especially from Python instincts)

- Coding before examples
- Silent assumption about sorted input
- Mutating inputs unexpectedly

Exercises

Work in `exercises/28-algorithm-drills/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/28-algorithm-drills/` until you have an honest failing attempt.

Self-check

1. What is your first step before code?
2. How do you pick a data structure?
3. When is brute force OK?

Further reading

- https://go.dev/blog/slices